

CHAPTER 6 — POINTERS AND VARIABLES

SLIDE 1 — WHAT IS A POINTER AND VARIABLE?

MEET POINTERS AND VARIABLES!

A **variable** stores a value.

A **pointer** stores the **address of a variable**.

Simple Definition

Variable → Stores a value 

Pointer → Stores the address of a variable 

LOOK AT THIS!

None

```
int x = 10;  
int *p = &x;
```

Here:

None

x → 10

x stores the value **10**.

None

p → address of **x**

`p` stores the **address** of `x`.

EASY ANALOGY

Think of a variable as a **box** .

None

 `x`
 `10`

The pointer is like a **note containing the location of that box** .

None

 `p` →  `x` → `10`

★ REMEMBER!

Variable → **VALUE**

Pointer → **ADDRESS**

SLIDE 2 — POINTER POINTING TO A VARIABLE

HOW DOES A POINTER POINT TO A VARIABLE?

First create a variable:

None

```
int x = 25;
```

Then create a pointer:

None

```
int *p = &x;
```

Here:

None

$x \rightarrow 25$

$p \rightarrow \text{address of } x$

WHAT DOES & DO?

The & operator gives the **address** of a variable.

None

$\&p$

would give the address of p .

But:

None

$\&x$

gives the address of x .

So:

None

```
int *p = &x;
```

means:

Store the address of x inside p .

EASY ANALOGY

Imagine your variable is a house .

None

$x \rightarrow$ 

The pointer doesn't store the house itself.

It stores the **house address** 📌.

None

p → 📌 Address of x

★ **REMEMBER!**

& → GET ADDRESS 📌

🟡 SLIDE 3 — 🔗 USING A POINTER TO ACCESS A VARIABLE

😞 **HOW CAN A POINTER ACCESS THE VARIABLE?**

We already know:

None

```
int x = 25;  
int *p = &x;
```

Now use:

None

*p

The * accesses the **value stored at the address**.

So:

None

p → address of x 📌
*p → value of x 🔍

Therefore:

None

```
printf("%d", *p);
```

Output:

None

```
25
```

LOOK AT THIS!

None

```
x
```

↓

```
25
```

```
p
```

↓

```
 address of x
```

```
*p
```

↓

```
25
```

EASY ANALOGY

Think of `p` as a **map** .

None

```
p → tells you WHERE 
```

```
*p → tells you WHAT is there 
```

MEMORY TRICK

```
& → ADDRESS 
```

```
* → VALUE 
```

★ TAKEAWAY

A pointer can be used to access the variable it points to.

SLIDE 4 — CHANGING A VARIABLE USING A POINTER

🤔 CAN WE CHANGE A VARIABLE USING ITS POINTER?

Yes! 🎯

Look at this:

None

```
int x = 10;  
int *p = &x;  
*p = 50;
```

Since `p` points to `x`:

None

```
Before:  
x = 10
```

After:

None

```
*p = 50
```

Now:

None

```
x = 50
```

LOOK AT THIS!

None

p



x



10

After:

None

*p = 50;

None

p



x



50

EASY ANALOGY

Think of p as a **key**  to a box.

None

p



x

Using:

None

*p = 50;

is like opening the box and replacing its contents.

BIG MEMORY TRICK

p → FIND THE VARIABLE 📌

***p** → ACCESS THE VALUE 🔍

***p = value** → CHANGE THE VALUE ✎

★ TAKEAWAY

A pointer can access and modify the variable it points to.

SLIDE 5 — POINTERS AND VARIABLES RECAP

🚀 WHAT DID WE LEARN?

1. VARIABLE

None

```
int x = 10;
```

x → Stores a value

📌 2. GET THE ADDRESS

None

```
&x
```

& → Gets the address of **x**

3. POINTER

None

```
int *p = &x;
```

p → Stores the address of **x**

4. ACCESS THE VALUE

None

***p**

***p** → Gets the value of **x**

5. CHANGE THE VALUE

None

***p = 20;**

This changes:

None

x = 20

ONE BIG MEMORY TRICK!

None

VARIABLE

↓

x

↓

VALUE

1	2
3	4

↑

*p

↑

POINTER

↓

p

↓

ADDRESS



Or simply:

$x \rightarrow \text{VALUE}$ 
 $\&x \rightarrow \text{ADDRESS}$ 
 $p \rightarrow \text{STORES ADDRESS}$ 
 $*p \rightarrow \text{ACCESS VALUE}$ 



QUESTIONS

1 What does a variable store?

- A) An address
- B) A value
- C) A pointer
- D) A function

 **Hint:** Think about `int x = 10;`.

 **Glossary:**

Variable $\rightarrow$ A named storage location that holds a value.

What does $\&x$ give?

- A) The value of x
- B) The address of x
- C) The value of p
- D) The address of p

 **Hint:** $\&$ is the address operator.

 **Glossary:**

Address operator $\&$ $\rightarrow$ Used to get the address of a variable.

2 What is the output?

None

```
int x = 25;  
int *p = &x;  
printf("%d", *p);
```

- A) 25
- B) Address of `x`
- C) 0
- D) `p`

 **Hint:** `*p` accesses the value of `x`.

 **Glossary:**

Value → The actual data stored in a variable.

3 Which statement correctly connects `p` to `x`?

- A) `int p = x;`
- B) `int *p = &x;`
- C) `int &p = x;`
- D) `int *p = x;`

 **Hint:** A pointer stores the address of the variable.

 **Glossary:**

Pointer declaration → Declaring a variable that stores an address.

4 What does `*p` give?

None

```
int x = 20;  
int *p = &x;
```

- A) Address of `x`
- B) Value of `x`
- C) Address of `p`
- D) Value `0`

 **Hint:** `*` is used to dereference a pointer.

 **Glossary:**

Dereference → Access the value stored at the address held by a pointer.

5 What does a pointer store?

- A) Only numbers
- B) A character

- C) The address of a variable
- D) A function

 **Hint:** A pointer tells us **where** a variable is stored.

 **Glossary:**

Pointer → A variable that stores a memory address.